

Flutter Mobile Developer Handover Guide

Remote Mandatory & Optional App Update Integration

Status: Implemented & Live on Dev Backend | API Version: v1

Core Architectural Principle: The backend owns and evaluates the update policy rules; the Flutter mobile application enforces and presents them. No minimum versions should be hard-coded in the mobile app binary. Operations and product teams remotely control version enforcement from the Admin Portal.

1. Public API Endpoint (No Authentication Required)

Call this endpoint during Flutter app startup and whenever the application resumes from the background:

```
GET /api/v1/app/update-config?platform=android&version=2.4.0
(Alias: GET /api/app/update-config)
```

Request Parameters:

Parameter	Type	Required	Description
platform	string	Yes	Client operating system: "android" or "ios" (lowercase)
version	string	Yes	Installed Flutter app SemVer string from package_info_plus (e.g. "2.4.0")
appld	string	No	Application flavor or bundle identifier (e.g. "com.oreedu.app")

2. JSON Response Contract

```
{
  "platform": "android",
  "currentVersion": "2.4.0",
  "minimumSupportedVersion": "2.5.0",
  "latestVersion": "2.6.0",
  "updateType": "force",
  "updateRequired": true,
  "title": "Update Required",
  "message": "A new version of the app is required to continue using the service.",
  "storeUrl": "https://play.google.com/store/apps/details?id=com.oreedu.app",
  "policyVersion": 2,
  "publishedAt": "2026-09-14T05:15:00.000Z"
}
```

Understanding updateType Actions:

updateType	updateRequired	Flutter Client Action
force	true	Block user completely. Display ForceUpdateScreen with back button disabled. Tapping "Update Now" opens storeUrl.
optional	false	Display optional update popup with "Update" and "Later" buttons. Allow user to continue into app.
none	false	Installed version is supported and up to date. Open normal application flow immediately.

3. Flutter Client Implementation Requirements

- 1. Root Startup Gate: Execute update verification before displaying login or loading authenticated application routes.
- 2. No Back-Button Bypass: Wrap ForceUpdateScreen in PopScope(canPop: false) (or WillPopScope) to prevent Android physical back button dismissal.
- 3. Lifecycle Awareness (App Resume Check): Use WidgetsBindingObserver to detect AppLifecycleState.resumed. When the user returns from Google Play / App Store after updating, re-run the update check to automatically unlock the app.
- 4. Fail-Open Safety Strategy: Set a 5-second HTTP timeout. If network is unavailable or backend times out, fallback to locally

- cached policy or allow normal entry (never leave user stuck on splash screen).
5. Store Launching: Use `url_launcher` with `LaunchMode.externalApplication` to open the dynamic `storeUrl` returned in the response.
 6. Deep Link Protection: Ensure deep links (e.g. push notifications) pass through the update gate before rendering target screens.

4. Complete Dart Data Model Snippet

Recommended implementation file: lib/core/app_update/app_update_config.dart

```
enum UpdateType { none, optional, force }

class AppUpdateConfig {
  final String platform;
  final String currentVersion;
  final String minimumSupportedVersion;
  final String latestVersion;
  final UpdateType updateType;
  final bool updateRequired;
  final String title;
  final String message;
  final String storeUrl;
  final int policyVersion;

  AppUpdateConfig({
    required this.platform,
    required this.currentVersion,
    required this.minimumSupportedVersion,
    required this.latestVersion,
    required this.updateType,
    required this.updateRequired,
    required this.title,
    required this.message,
    required this.storeUrl,
    required this.policyVersion,
  });

  factory AppUpdateConfig.fromJson(Map<String, dynamic> json) {
    UpdateType parseType(String? type) {
      switch (type?.toLowerCase()) {
        case 'force': return UpdateType.force;
        case 'optional': return UpdateType.optional;
        default: return UpdateType.none;
      }
    }

    return AppUpdateConfig(
      platform: json['platform'] ?? '',
      currentVersion: json['currentVersion'] ?? '',
      minimumSupportedVersion: json['minimumSupportedVersion'] ?? '1.0.0',
      latestVersion: json['latestVersion'] ?? '1.0.0',
      updateType: parseType(json['updateType']),
      updateRequired: json['updateRequired'] ?? false,
      title: json['title'] ?? 'Update Required',
      message: json['message'] ?? 'A new version of the app is available.',
      storeUrl: json['storeUrl'] ?? '',
      policyVersion: json['policyVersion'] ?? 1,
    );
  }
}
```

5. Store Launcher Helper

Recommended implementation file: lib/core/app_update/store_launcher.dart

```
import 'package:url_launcher/url_launcher.dart';

Future<void> openStore(String url) async {
  if (url.isEmpty) return;
  final uri = Uri.parse(url);
  if (await canLaunchUrl(uri)) {
    await launchUrl(uri, mode: LaunchMode.externalApplication);
  }
}
```

6. Admin Portal Controls & Workflows

The operations and release management teams control update policies via the Admin Portal at /mobile-updates:

- Platform-Specific Tabs: Android and iOS have completely independent version rules and store URLs.
- Version Management: Set Minimum Supported Version (e.g. 2.5.0) and Latest Released Version (e.g. 2.6.0).
- Enforcement Modes: Switch between None, Optional, and Force Update.
- Dynamic Messaging: Customize user-facing dialog title and description without code changes.
- Interactive Live Simulator: Built-in tester lets admins test any version number and preview the exact response and phone screen mockup.
- Safety Confirmation Prompt: Prevents accidental lockouts by requiring confirmation that new builds are live on store before publishing a force update.

7. Integration Checklist for Flutter Developer

- [] Add package_info_plus and url_launcher dependencies in pubspec.yaml
- [] Implement AppUpdateConfig model and JSON parsing
- [] Create AppUpdateService to call GET /api/v1/app/update-config
- [] Build ForceUpdateScreen (back button disabled, "Update Now" button only)
- [] Build OptionalUpdateDialog ("Update" and "Later" buttons)
- [] Register WidgetsBindingObserver to re-evaluate policy on AppLifecycleState.resumed
- [] Implement 5-second timeout and offline fail-open fallback
- [] Test with ?platform=android&version=0.9.0 (verify Force Update screen triggers)
- [] Test with ?platform=android&version=2.6.0 (verify Allowed seamless entry)